

MAIA

Intelligent Monitoring & Auto-Healing Platform

Your AI Monitoring Assistant for Intelligent Operations

System Specification Document
Version 1.0 · May 2026

Executive Summary

MAIA is an intelligent monitoring and auto-healing platform for automated job pipelines (DTSX/SSIS). It continuously watches for failures, classifies them by type, generates fix recommendations, and either resolves them automatically or routes them to an operator for approval — all through a modern web-based dashboard.

■ Faster Detection	■ Less Manual Work	■ Full Audit Trail	■ Self-Service Config
Seconds vs. hours of manual log review	Configurable auto-heal rules reduce toil	Every action recorded immutably	No database access needed for config

System Architecture

MAIA watches your pipelines, understands what went wrong, knows how to fix it, and either fixes it automatically or tells the operator exactly what to do.

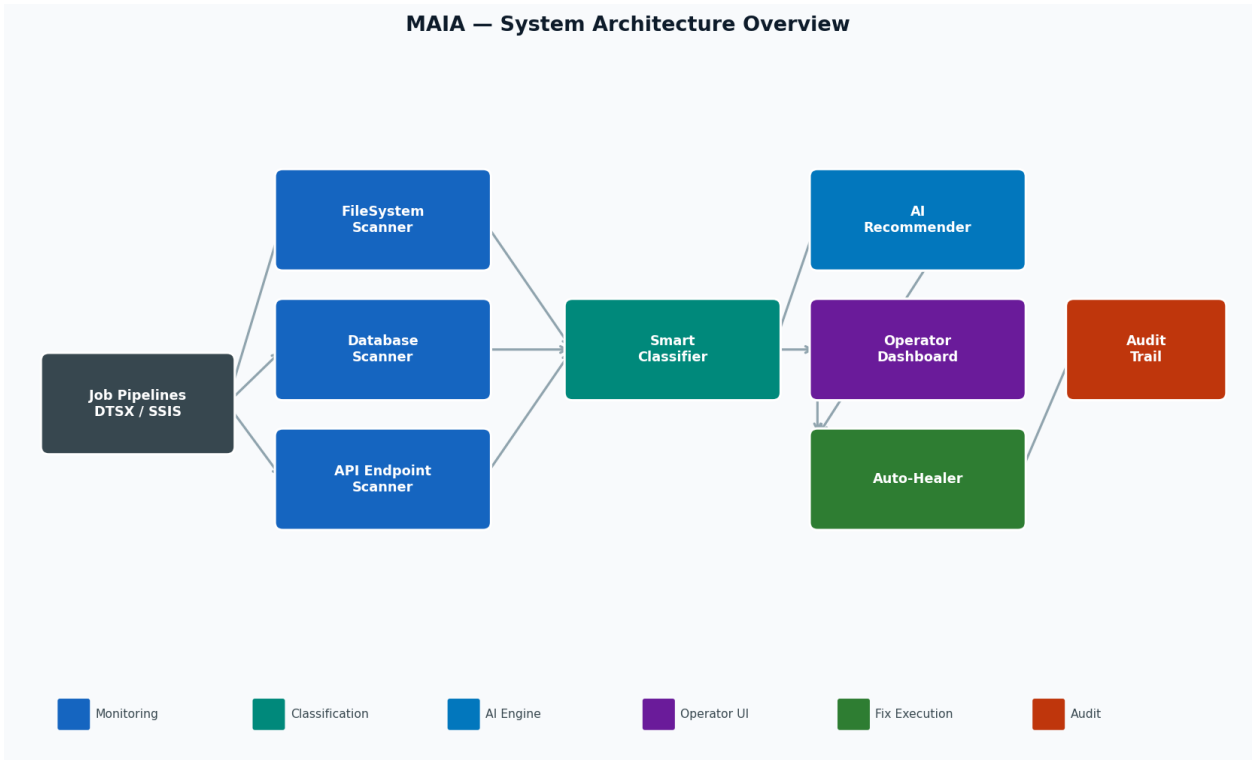


Figure 1 — MAIA System Architecture

Key Stakeholders & User Roles

Role	Who They Are	What They Do
Operator	Data / Support Engineer	Reviews failures, approves fixes, sets auto-heal rules
Admin	Senior Engineer / Team Lead	Configures monitored jobs, classification and fix policies
System (Auto)	MAIA itself	Runs scans, classifies failures, executes auto-heal fixes
Auditor	Compliance / Management	Reviews the immutable audit log for compliance

Operator Workflow

The primary day-to-day workflow for an operator responding to a pipeline failure.

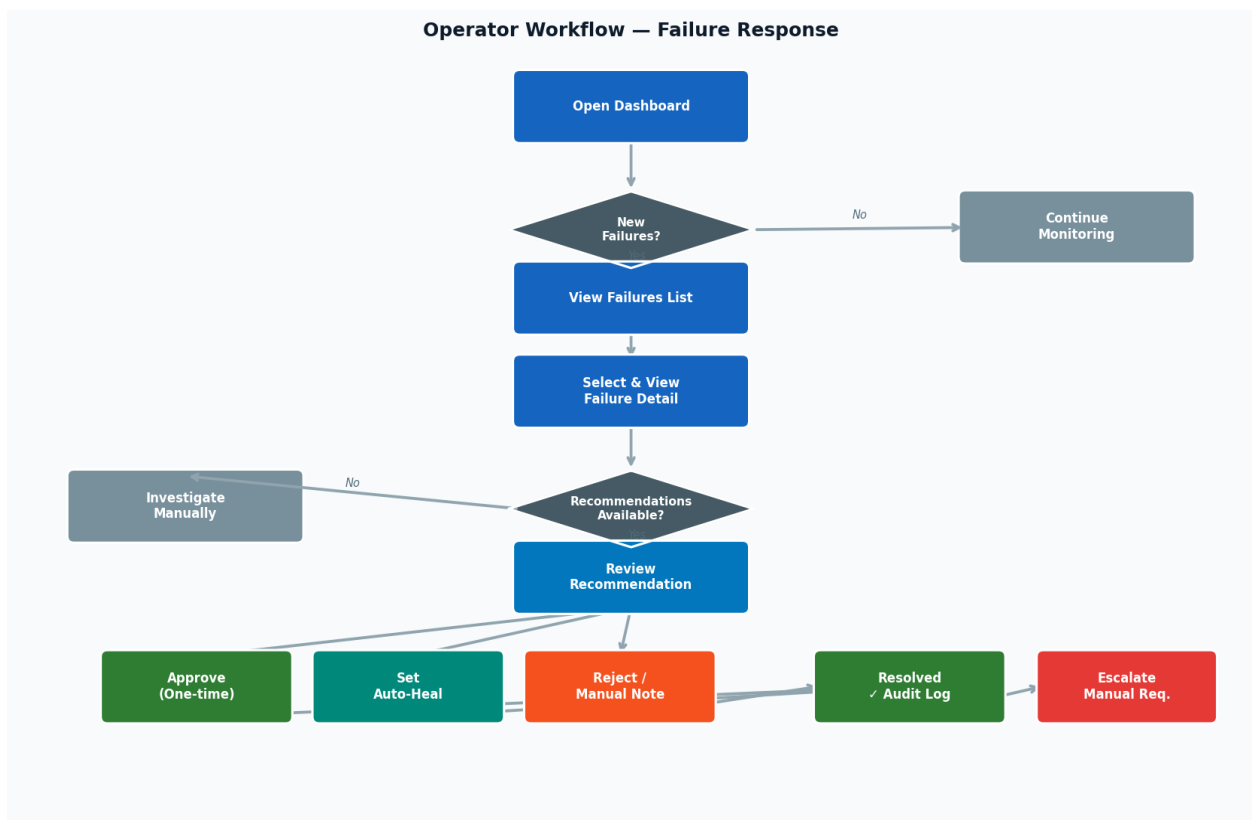


Figure 2 — Operator Failure Response Workflow

Auto-Heal Workflow

Once auto-heal rules are configured, MAIA resolves failures without any human intervention. The background worker runs every 60 seconds.

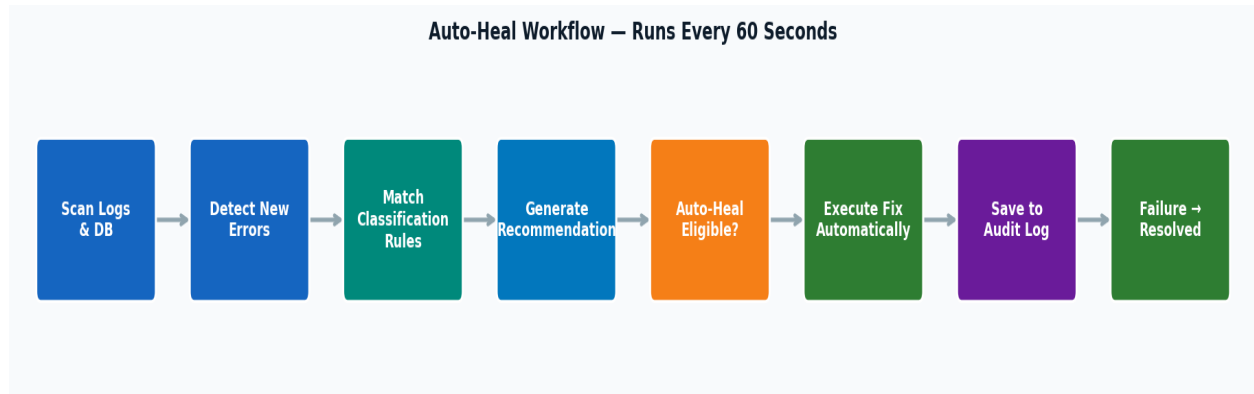


Figure 3 — Auto-Heal Background Process

Auto-Heal is Enabled When...

An operator previously approved a recommendation and toggled "Set as Auto-Heal"

A Fix Policy Rule for that error type has IsAutoHealEligible = true configured by an admin

Configuration Management Workflow

How an admin or configuration manager sets up a new job for monitoring through the web UI — no database access required.

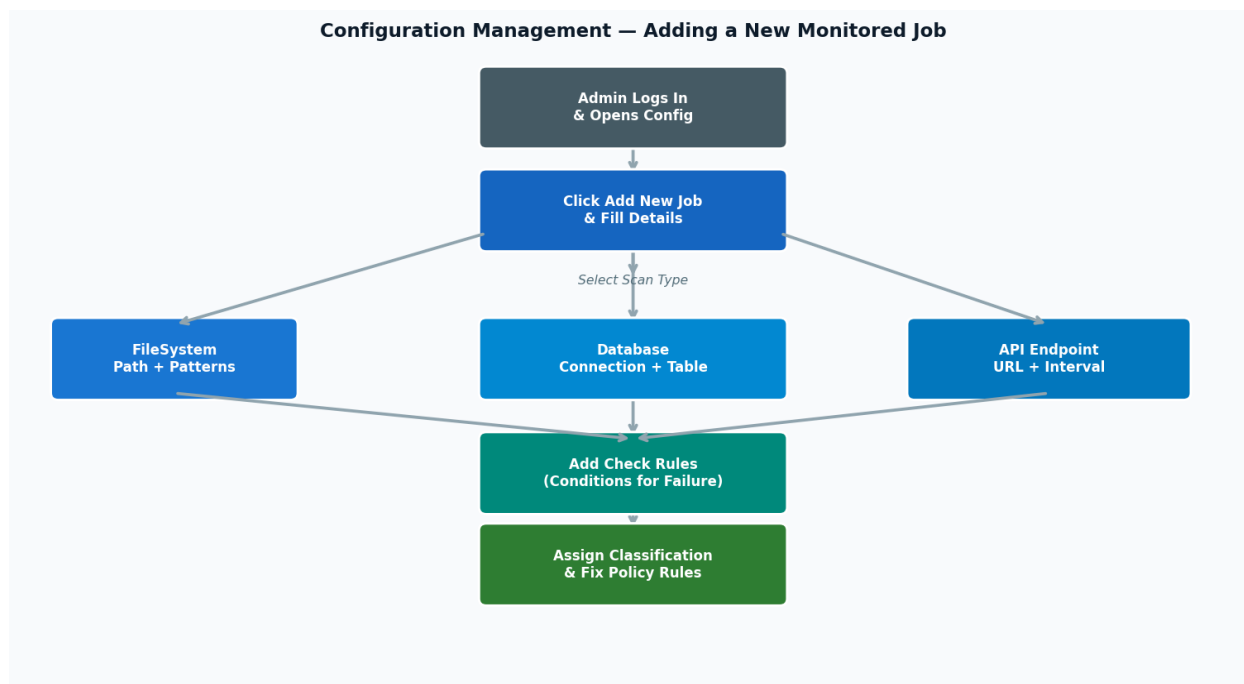


Figure 4 — New Job Configuration Workflow

Feature Status

Feature Status Overview	
Monitoring	
FileSystem scan	✓ Built
Database scan	✓ Built
API endpoint scan	✓ Built
Incremental watermarks	✓ Built
Configurable polling interval	✓ Built
Classification	
Regex rule matching	✓ Built
Per-job rule overrides	✓ Built
Confidence scoring	✓ Built
Job type & error type tagging	✓ Built
Fix Execution	
Rule-based recommendations	✓ Built
HTTP API call executor	✓ Built
Stored procedure executor	✓ Built
Script executor (120s timeout)	✓ Built
Manual action handler	✓ Built
Operator UI	
Failures dashboard	✓ Built
Failure detail view	✓ Built
Recommendations screen	✓ Built
Operator action history	✓ Built
MonitoredJob config forms	🔄 In Progress
Auto-heal toggle on recs	🔄 In Progress
Fix policy rules UI	🔄 In Progress

Figure 5 — Feature Build Status

Glossary

Term	Definition
MonitoredJob	A configured pipeline that MAIA watches. Has scan type, polling interval, and rules.
ScanCheckRule	A condition evaluated during a DB scan (e.g. "row count must be > 100").
ClassificationRule	A regex pattern that assigns a JobType and ErrorType to a failure.
JobFailure	A detected failure instance — one error event from a pipeline.
AiRecommendation	A suggested fix action with confidence score and auto-heal eligibility.
FixPolicyRule	Admin-configured rule: for ErrorType X on JobType Y, run this action.
Auto-Heal	Automatic fix execution without operator approval, governed by IsAutoHealEligible.
OperatorAction	A human decision recorded — approve, reject, or set auto-heal.
FixExecutionLog	Result record of a fix attempt: success/failure, timestamp, trigger type.
AuditLog	Immutable append-only log of all significant system actions.
Watermark	Saved position (file offset or DB row ID) enabling incremental scanning.
Confidence Score	A 0.0-1.0 value indicating certainty of a classification match or suggestion.
FixCategory	Broad fix category: Retry, FileRepair, DbFix, or Manual.
DTSX	File format for SQL Server Integration Services (SSIS) packages.

TECHNICAL APPENDIX

Architecture · Workers · Data Flow · Fix Engine · Scan Strategies · API Catalog

Appendix A — Clean Architecture Layers

MAIA follows Clean Architecture — all dependencies point inward only. The Core domain has zero external dependencies; Infrastructure implements Core interfaces; the API layer orchestrates use cases without touching the database directly.

Layer	Components	Responsibility
Presentation (AIEngineAPI)	DataController · ConfigController ClassificationController · FixController PipelineController · JobScanController GlobalExceptionHandler	HTTP endpoints, request routing, error formatting (RFC 7807)
Application (Use Cases)	ClassifyJobsUseCase GenerateSuggestionsUseCase ExecuteFixesUseCase DirectoryPipelineUseCase	Orchestrate domain logic, no infrastructure dependencies
Core (Domain)	Entities: JobFailure · MonitoredJob AiRecommendation · FixPolicyRule Interfaces: IFixEngine · IScanStrategy Enums: FixActionType · ScanType	Business rules & contracts, zero external dependencies
Infrastructure	SqlRepositories (9x) · Scan Strategies Fix Executors · RuleBasedClassifier Background Workers (4x)	Data access, external calls, implements Core interfaces

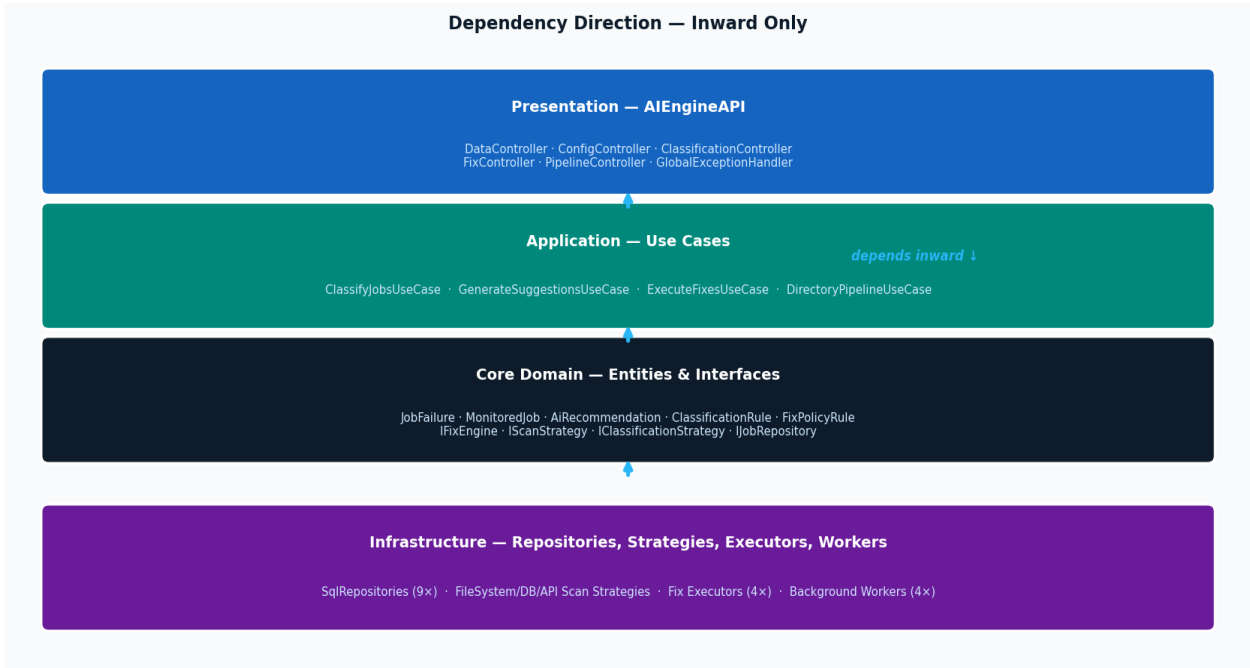


Figure A1 — Clean Architecture Layer Dependency

Appendix B — Background Worker Pipeline

MAIA runs four background workers. The MonitoringWorker drives the complete pipeline every 60 seconds. The other three workers can run the same use cases independently for targeted operations (classify-only, parse-only, suggest-only).

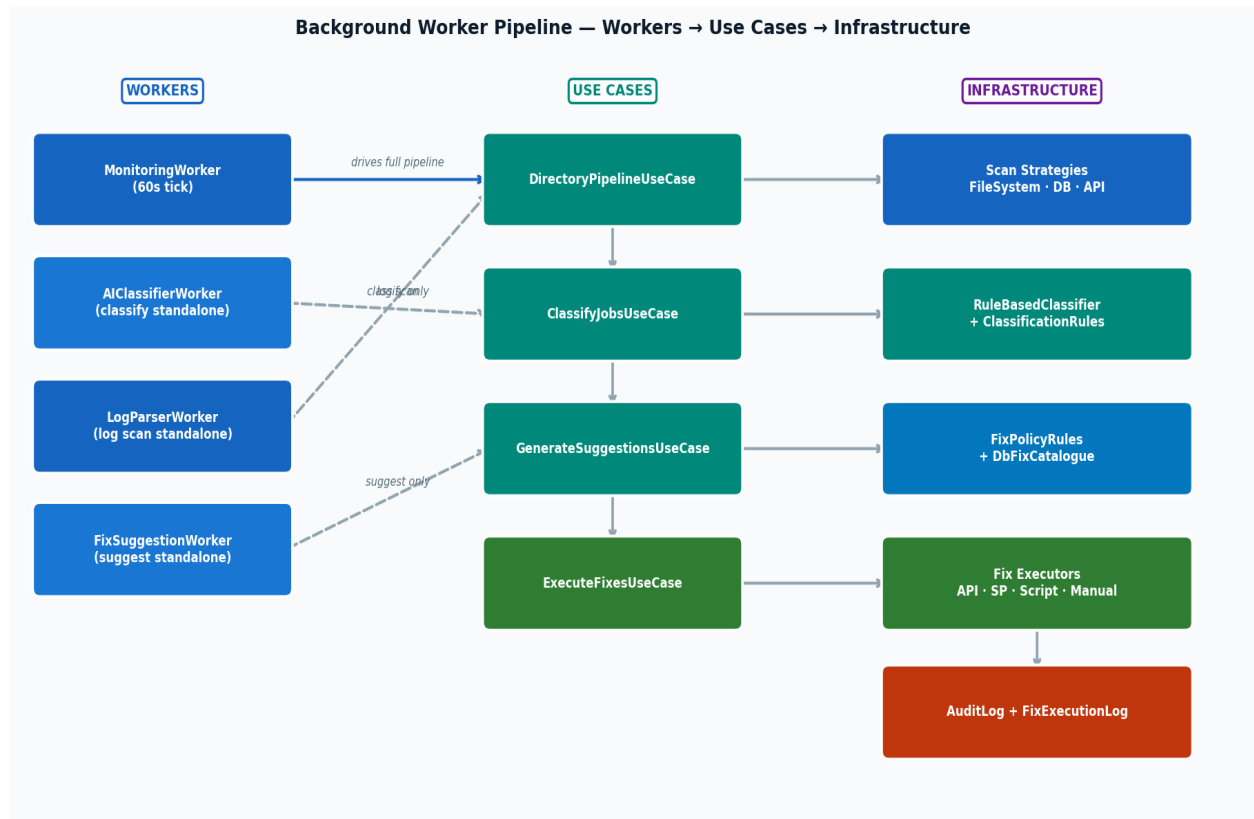


Figure B1 — Background Worker Pipeline & Use Case Orchestration

Appendix C — End-to-End Data Flow

Traces the complete journey of a failure event — from pipeline detection through classification, recommendation generation, and fix execution to final resolution.

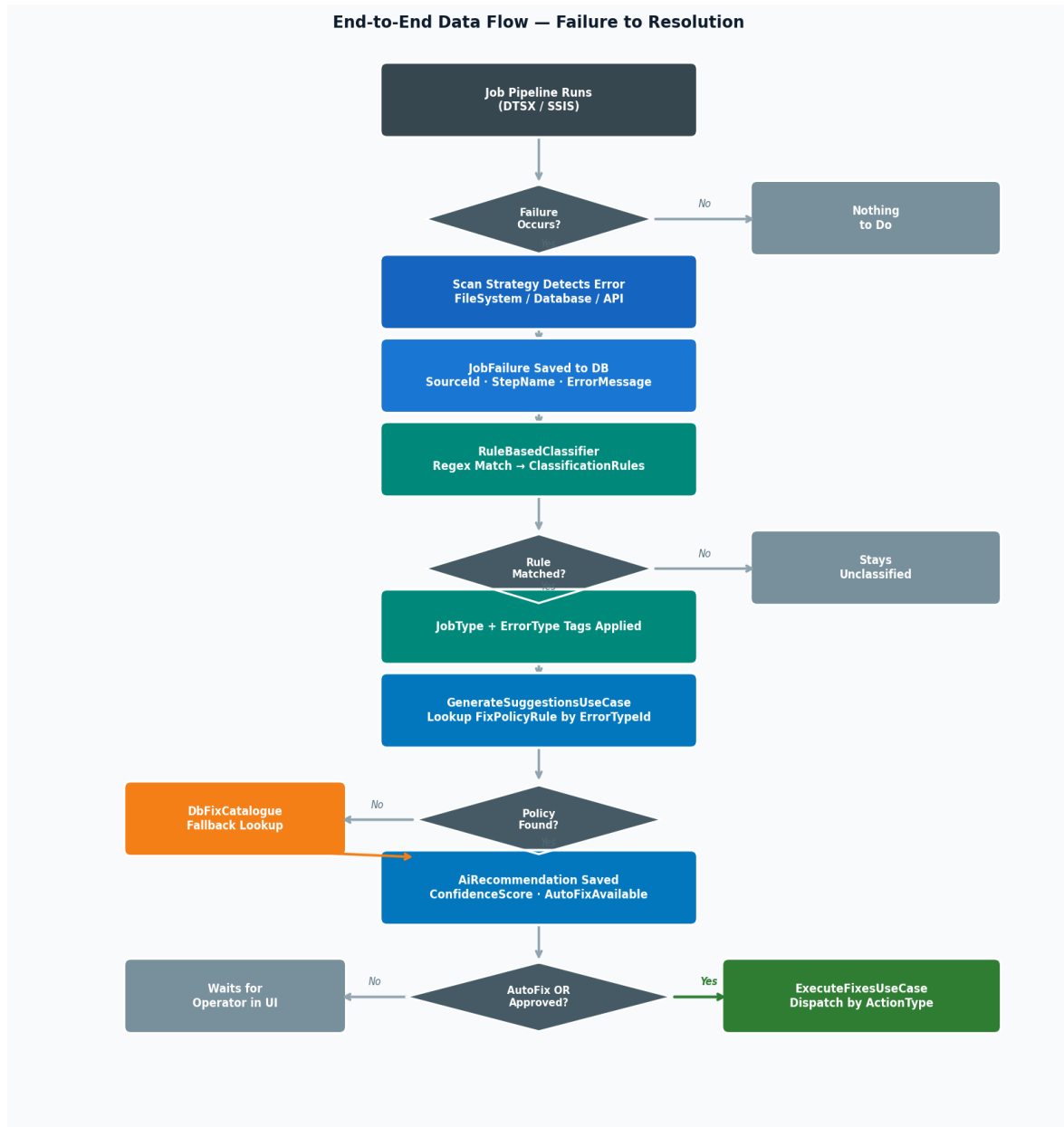


Figure C1 — End-to-End Data Flow: Pipeline Failure to Resolution

Appendix D — Fix Execution Engine

When a fix is approved (manually or via auto-heal), the ExecuteFixesUseCase dispatches to the appropriate executor based on the FixPolicyRule ActionType. If no policy exists, it falls back to category-based handlers.

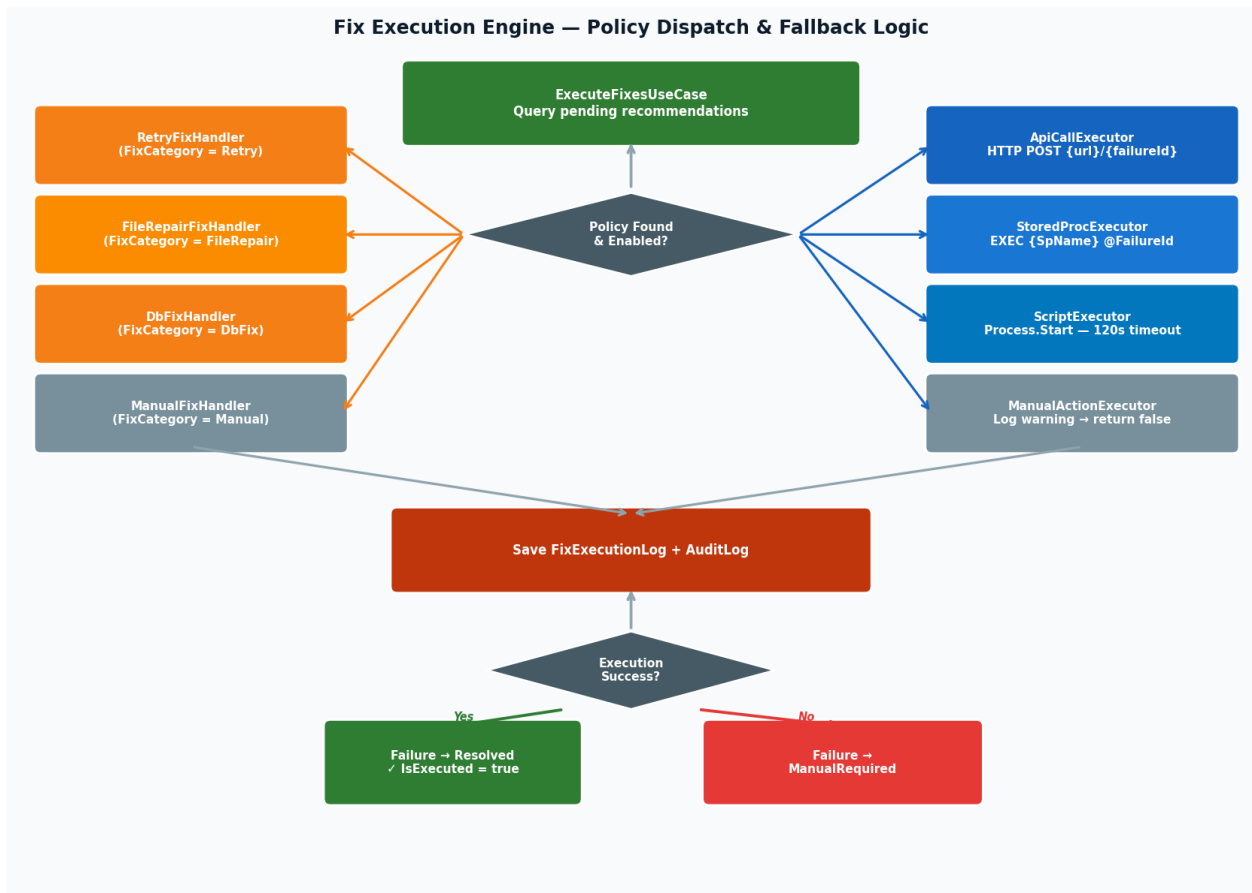


Figure D1 — Fix Execution Engine: Policy Dispatch & Fallback Handlers

Appendix E — Scan Strategies

Each MonitoredJob is configured with one of three scan strategies. The strategy is selected at runtime based on the ScanType field. All strategies save discovered failures to SQL Server and use watermarks to prevent re-processing.

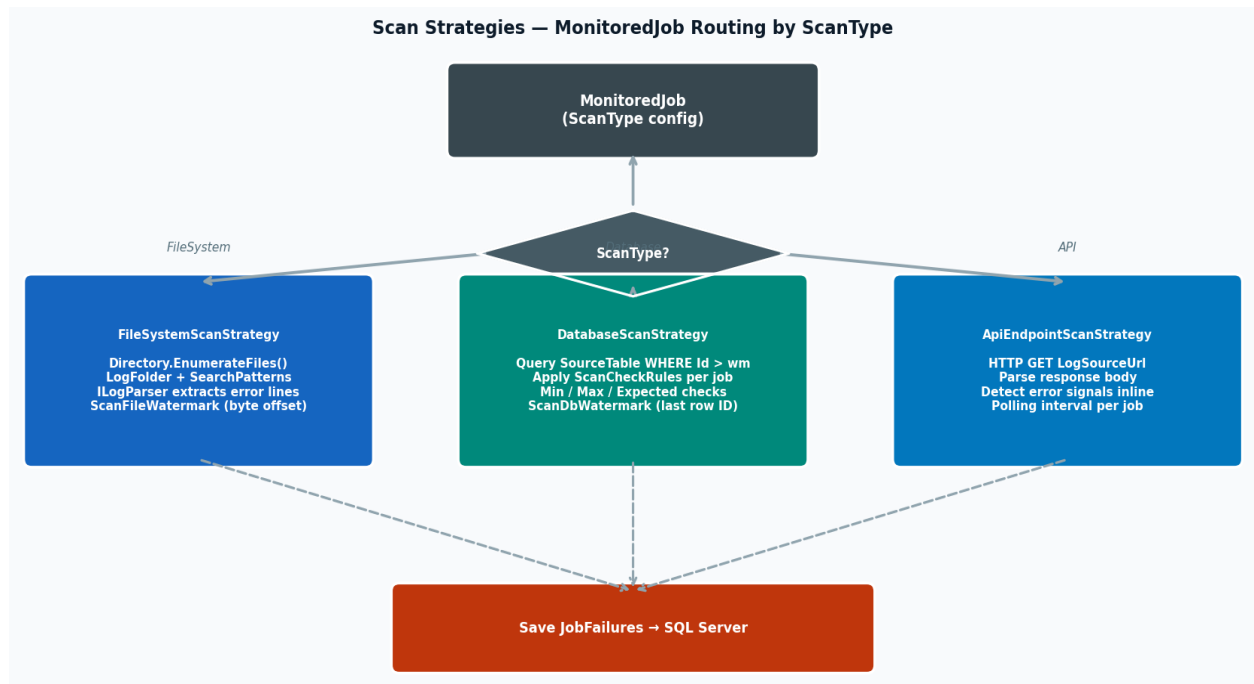


Figure E1 — Scan Strategy Selection & Watermark Tracking

Strategy	Input	Watermark	Check Rules
FileSystemScanStrategy	Log files in LogFolder (SearchPatterns e.g. *.log)	ScanFileWatermark (byte offset)	N/A — parses error lines directly
DatabaseScanStrategy	SourceTable rows WHERE Id > watermark	ScanDbWatermark (last processed row ID)	ScanCheckRules (Min/Max/Expected)
ApiEndpointScanStrategy	HTTP GET response from LogSourceUrl	Per polling interval (no persistent watermark)	Response body error signal detection

Appendix F — API Endpoint Catalog

All HTTP endpoints exposed by AIEngineAPI. The API is organized into controllers by concern — read-only dashboard data, CRUD configuration, and pipeline processing triggers.

DataController — Read-Only Dashboard Queries

Method	Endpoint	Description	Response
GET	/api/data/failures	Paginated job failures	PagedResult<JobFailureDto>;
GET	/api/data/failures/{id}/status	Failure detail + linked recommendations	FailureStatusDto
GET	/api/data/recommendations	Paginated AI recommendations	PagedResult<RecommendationDto>;
GET	/api/data/monitored-jobs	All active monitored jobs with rules	MonitoredJobDto[]

ConfigController — CRUD Configuration Endpoints

Method	Endpoint	Description	Response
CRUD	/api/config/monitored-jobs	Manage MonitoredJob entities	GET/POST/PUT/DELETE
CRUD	/api/config/scan-check-rules	Manage ScanCheckRules per job	GET/POST/PUT/DELETE
CRUD	/api/config/classification-rules	Manage ClassificationRules	GET/POST/PUT/DELETE
CRUD	/api/config/fix-policy-rules	Manage FixPolicyRules	GET/POST/PUT/DELETE

Processing Controllers — Pipeline Triggers

Method	Endpoint	Description	Response
POST	/api/classification/classify	Run classification on pending failures	200 OK / ProblemDetails
POST	/api/fix/execute-fixes	Execute approved / auto-heal fixes	200 OK / ProblemDetails
POST	/api/pipeline/run-pipeline	Run full directory pipeline	200 OK / ProblemDetails
POST	/api/process/process	Full pipeline via all use cases	200 OK / ProblemDetails
POST	/api/logparser/parse	Parse a log file on demand	200 OK / ProblemDetails

Appendix G — Non-Functional Requirements

Concern	Approach
---------	----------

Reliability	MonitoringWorker restarts on crash (hosted service); AuditLog is append-only and immutable
Offline Support	Rules and config stored locally in SQL Server; no cloud dependency at runtime
Scalability	Worker projects (AIClassifierWorker etc.) can run as separate services or containers
Auditability	Every fix execution writes to FixExecutionLog and immutable AuditLog
Error Handling	GlobalExceptionHandler returns RFC 7807 ProblemDetails on all unhandled exceptions
Extensibility	New scan types: implement IScanStrategy. New fix types: implement IFixActionExecutor
Incremental Scanning	ScanDbWatermarks / ScanFileWatermarks prevent re-processing already-seen records
Confidence Scoring	ClassificationRules carry Confidence (0.0–1.0); recommendations inherit the score